

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

SLOT-D

Course Code:	CSA1407	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	P. Pallavi	Reg. No.:	192525448
Section / Batch:	B.Tech (NISM)	Date:	16/04/2026

Code of Conduct

I, P. Pallavi (192525448) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Pallavi

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	5 / 4	4 / 4	4 / 4	3 / 4	91.25
2	4 / 4	4 / 4	3 / 4	4 / 4	4 / 4	98.75
3	3 / 4	4 / 4	3 / 4	4 / 4	3 / 4	85
4	4 / 4	4 / 4	3 / 4	3 / 4	4 / 4	88.75
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	89.75
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. P. H

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) $(\epsilon + aa)^*$ b) $(a + b)^*$ abb c) $(a + b)^*$ aba $(a + b)^*$ d) $(a + ab)^*$ e) $(aba)^* + (a + b)^*$
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of $\{0,1\}^*$: i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1's.
5	Consider a lexical analyzer for a simplified programming language. The language

consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, ,,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

```
int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }
```

Questions:

- a) How many tokens are there in the given input string?
 - b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
 - c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.
-

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear or interpretation	No interpretation

	Level 4 - Proficient	Level 3 - Developing	Level 2 - Beginning	Level 1 - Misinterprets or unable to understand problem
Early interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding misses elements	Inappropriate or partially correct method	Unable to select method
Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Mostly correct steps with minor mistakes	Disorganized steps; errors	No process
Logical, systematic steps with correct approach throughout	Minor calculation errors	Frequent incorrect answer	errors; final	No calculations
All calculations accurate; correct final answer	Basic interpretation with minor omissions	Limited unclear interpretation	or	No interpretation
5%	Clearly interprets results with units and engineering relevance			

CSA1407 - Compiler Design

Name: P. Pallavi

RegNo: 1925254118

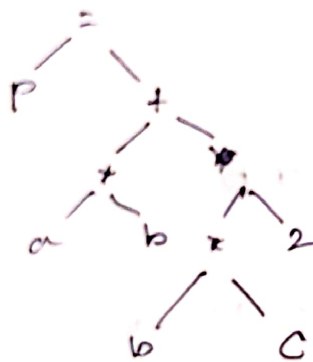
Trace the processing of the expression
Given Expression

$$P: (a * b) + (b * c) * 2$$

(a) Tokens generated during Lexical Analysis:

$P, =, (, a, *, b,), +, (, b, *, c,), *, 2, \text{EOF}$

(b) Syntax tree:



(c) Intermediate Code:

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

(d) Optimized code:

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_2 = t_2 * 2$$

$$P = t_1 + t_2$$

Significance :

Executable by the processor
faster due to optimization
produces efficient machine code.

2) Language for Regular Expressions

(a) Regular expression:

$$(\epsilon + aa^*)^*$$

Language:

all strings consisting only of a's (including empty string)

Eg: $\epsilon, a, aa, aaa, aaaa$

(b) Regular expression:

$$(a^* + b^*)^* abb$$

Language:

Any combination of a's and b's ending with

eg: $abb, aabb, babb$

(c) Regular Expression:

$$(a^* b^*)^* aba (a^* + b^*)^*$$

Language: strings containing data

Eg: $aba, aabaa$

(d) Regular expression: $(a + ab)^*$

Language: strings made by repeating a or ab

Eg: $\epsilon, a, ab, aa, abab$

Regular Expression:

$$(aba)^* + (a^*b^*)^*$$

Example: Either repetitions of aba or any combination of consecutive a's followed by b's

Eg: ϵ , aba, aaabbb, aab

(i) Number of Tokens

main.c

```
int a=10;  
char b="abc";  
int c=30;  
char d="xyz";  
int m=40.5;  
}
```

The tokens are

main.c	4
{	1
int a=10;	5
char b="abc";	5
int c=30;	5
char d="xyz";	5
int m=40.5;	5
}	1

Total tokens are 31

(i) Lexical Errors:

1. 1xab is not a valid integer constant
2. Missing

4) Regular Expressions

(i) Even number of 0's

$$1^*(01^*01^*)^*$$

(ii) At least one 1

$$(0+1)^* 1 (0+1)^*$$

(iii) odd number of 0s and odd number of 1s

$$(00)^* 1 (11)^* 00^* 1 (00)^*$$

(iv) Starts with 1 and ends with 0

$$1(0+1)^* 0$$

(v) No Consecutive 1s

$$(0+10)^* 1?$$

5) Lexical Analyzer

(i) Num of Tokens; Total tokens are 20

int, float, if, return $\rightarrow$ keyword

=, == $\rightarrow$ operator

12345, 123.45, $\rightarrow$ Integral Literal

{, }, {, } $\rightarrow$ Separator

1234.5 $\rightarrow$ Floating Literal

(ii) Categorized Tokens

keywords	Operators	Separators	identifiers	Integral Literal
int	=	{, {, }	var123	12345
float	==	{, {	var546	
if			var123	
return			var456	

(iii) Estimated FSM state Transitions

keywords (4)

18

identifiers (4)

28

operators (3)

4

separators (1)

7

Floating Literal

8

Integer Literal

6

Total estimated transitions

$$18 + 28 + 4 + 7 + 6 + 8 = 71$$